

A study in the optimality of adaptable sorting: MergeCearense

Victor Almeida Campos

Théo Araújo Magalhães

Rafael de Paiva Lima Filho

October 6, 2025

Abstract

We propose a different way to implement natural mergesorts: as a trie-tree of indexes. The trie-tree interpretation helps us define a different view of a famous sorting algorithm, Powersort, as well as come up with an improved version: MergeCearense. We demonstrate ¹ that our method is competitive and that our trie-tree view helps explain and better visualize optimal merge run strategies.

1 Introduction

Theo: Sorry I ain't writing this now.

1.1 Outline

Theo: tem um nome melhor pra essa seção, a ideia é servir um sumário pra cada seção do artigo. esqueci o nome.

¹(Hopefully)

2 Context

The study of sorting algorithms has become a rather complex and dense endeavor, with some papers leaning heavily on previously established proofs, definitions and ideas. That is the burden of a well established area of study that also has a great amount of intersection with other problems.² We aim, within this and further sections, to define all of the concepts necessary to understand this paper, as to avoid publishing work that requires extensive complementary reading.

2.1 Natural Mergesort

Mergesort is a well-known sorting algorithm that consists of using divide-and-conquer techniques to sort an array.

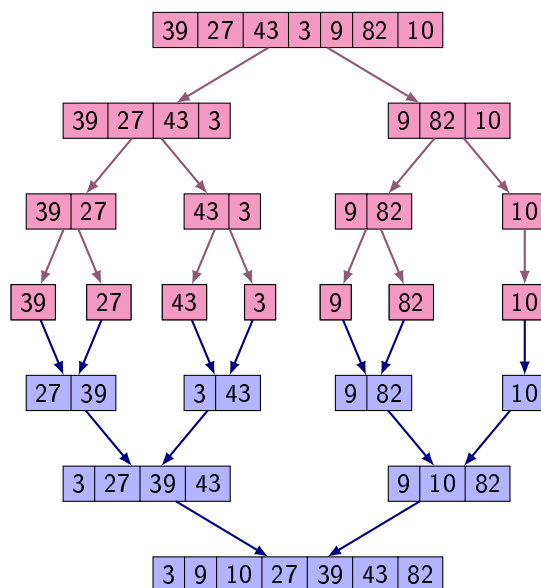


Figure 1: Mergesort tree with 7 elements

The figure above shows us the functioning of the algorithm: It descends until each subarray has size one, merging it with its neighbor. All subsequent subproblems are solved by merging a subarray with its neighbor until the entire array is sorted. If you look closely at the MergeSort tree above, though, you can see that the subproblem $[9, 82]$ is already solved: $[9, 82]$ is a sorted array. This is what we call a run.

Definition 1. A **Run** is a contiguous region of the input that is non-decreasing or non-increasing, i.e it already is in sorted order or it can be reversed to be in sorted order. For mergesort, it is a subproblem that is already solved.

After defining runs, it is natural to think about engineering a MergeSort algorithm that takes them into consideration. This is what we call a natural merge sort.

Definition 2. A **Natural Mergesort** is a sorting algorithm that takes advantage of existing runs in the input to perform merges.

Theo: to-do: Adaptability, the advantage of looking into runs (i.e adapting to entropy), defining entropy, talking about other sorting algorithms. The idea here is to start as if doing a historical recap of sorting, but jumping directly to the study of natural mergesorts. Here, I want to talk about Timsort, Huffman, the algorithm that uses Huffman trees just to give a general idea of what has been done already. Then, we transition to powersort and talk about optimality and the idea of a merge tree to define merge strategies/policies. While talking about timsort we introduce the idea of a merge policy. Here, we talk about merge costs. I don't want to make this section just a throwing of information, though, which is why im trying to give it some 'flow'. Infodump = bad.

²Searching and Sorting, for example, are intimately tied to each other. [Citation needed]

2.2 Trie Trees

Due to the nature of natural mergesorts, this article will mention and define a variety of tree-like datastructures. Previously, we have shown the structure of a MergeSort Tree for the standard MergeSort algorithm, now, we will define the datastructure that will serve as the basis for our natural mergesort algorithm.

Definition 3. A **Trie** is an M-ary tree, whose nodes are M-place vectors with components corresponding to digits or characters. ³

For the purposes of this article, we will be dealing with 2-ary trees whose nodes are labelled as either '0' or '1'.

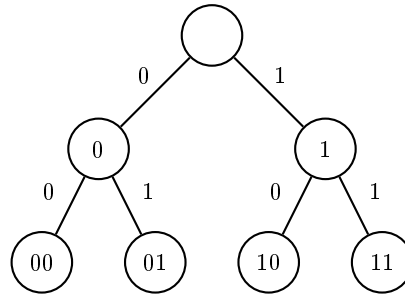


Figure 2: 2-ary Trie Tree

In the figure above, one might notice that we have labelled not just the nodes, but also the edges of our tree. It is also possible to observe that '0' means 'descending leftways' and '1' means 'descending rightways'. Tries have applications in the realm of string-searching (specifically web searching), data-mining, internet routing, among others.

Theo: THE FOLLOWING EXPLANATION IS A TO-DO!!!! NOT FINISHED!!! THIS SEGMENT WILL MOST LIKELY NOT BE ENTIRELY FOUND HERE.

For the purpose of our work, we apply Trie Trees to find the lowest common ancestor of two adjacent runs by giving each run indexes that will fit in our desired Trie Tree. Our Trie Tree goes through a 'compression' step that resolves 'useless' levels, raising all runs to their first divergence node. This is the tree that we will use for our MergeSort algorithm and our merge policy will be the same as that of PowerSort, given that we have the same guarantee of Indexes.

Theo: BELOW SEGMENT IS A PLANNING LIST

I think if we previously define Huffman Trees and how people use those trees for merges we can segway into talking the indexes for trie-trees and compressed trees. Its better to work on the previous context than to continue talking about trie trees. Trie trees are slightly novel, i think, for most people, so difining other things to compare them with seems better than just jumping right into talking about them.

3 Powersort and optimality

Theo: nas nossas issues, tem primeiro falar da cartesian tree e depois do powersort. mas hmm acho que isso fica meio estranho pq o powersort que chega com a ideia de cartesian trees. na seção anterior (de trie trees) a gente pode mostrar que as trie trees garantem o limite de entropia se os índices forem bons sem falar de cartesian trees e aí trazer as cartesian trees aqui. Essa seção vai ser grande i guess

³referenced from the art of computer programming sorting and searching page 492.

- 3.1 Trie-Tree Powersort
- 4 Mergesort Cearense
- 5 Running-Time study
- 6 Conclusion and future work
- 7 Implementations